

$$tf_{t,d} = \frac{f_{t,d}}{\sum_k f_{k,d}}$$

$$idf_t = \log \frac{N}{1 + df_t}$$

$$tfidf_{t,d} = tf_{t,d} \times idf_t$$



$$P(c|d) = \frac{P(c) \prod_{i=1}^n P(w_i|c)}{P(d)}$$

$P(c|d)$ = posterior probability of class c given document d

$P(c)$ = prior probability of class c

$P(w_i|c)$ = likelihood of word w_i given class c

$P(d)$ = probability of document d

$f_{t,d}$ = frequency of term t in document d

df_t = number of documents containing term t

N = total number of documents

$$\hat{c} = \underset{c}{\operatorname{argmax}} P(c|d)$$



NEUROVA AI

AI SOLUTIONS

NLP Training

Project 1

```
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
```

```
def preprocess(text):
    tokens = nltk.word_tokenize(text.lower())
    tokens = [w for w in tokens
               if w.isalnum()]
    tokens = [w for w in tokens
               if w not in stopwords.words('english')]
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(w) for w in tokens]
    return tokens
```

$$P(x_i|c) = \frac{1}{\sqrt{2\pi\sigma_{ic}^2}} e^{-\frac{(x_i - \mu_{ic})^2}{2\sigma_{ic}^2}}$$

μ_{ic} = mean of feature x_i in class c

σ_{ic}^2 = variance of feature x_i in class c

```
# Naive Bayes with NLTK
from nltk.classify import NaiveBayesClassifier
from nltk.classify.util import accuracy

# Assume features is list of (feature_dict, label)
train_set, test_set = featuresets[100:], featuresets[:100]
classifier = NaiveBayesClassifier.train(train_set)
print("Accuracy:", accuracy(classifier, test_set))

# Classify new instance
new_doc = {'good': True, 'movie': True, 'boring': False}
print(classifier.classify(new_doc))
```

language

corpus

sentiment

tokenization

text

analysis

classification

model

senbedding

vector

Data sets

Arabic Data set

<https://www.kaggle.com/datasets/mohamedramadan2040/arabic-customer-reviews>

English Data set

<https://www.kaggle.com/datasets/mwallerphunware/imbd-movie-reviews-for-binary-sentiment-analysis>

This project will be divided into **two main stages** and aims to build a complete NLP pipeline capable of detecting the language of a user-provided text and then performing sentiment analysis using a **language-specific model**. The project is designed specifically to practice the concepts covered in **Classical NLP**.

Stage 1: Language Detection

The first stage will focus on identifying whether the text entered by the user is written in Arabic or English.

you must first build an appropriate text preprocessing pipeline before training or applying their language-classification solution. The preprocessing pipeline should prepare the raw text for classical NLP techniques

After preprocessing, students must extract meaningful features from the text using classical feature-extraction techniques, such as Bag of Words, TF-IDF, or n-gram representations.

The language detector must then use a classical machine-learning classification algorithm, such as Naive Bayes, Logistic Regression, SVM, or another appropriate classical classifier.

Stage 2: Language-Specific Sentiment Analysis

Once the language has been detected, the system should route the text to the appropriate sentiment-analysis pipeline.

- If the detected language is Arabic, the text must be processed using an Arabic sentiment-analysis model.
- If the detected language is **English**, the text must be processed using an English sentiment-analysis model.

Both sentiment-analysis solutions must also follow the **classical NLP approach**.

you should build their own preprocessing and feature-extraction pipelines and use classical machine-learning algorithms for sentiment classification

Required Final Output

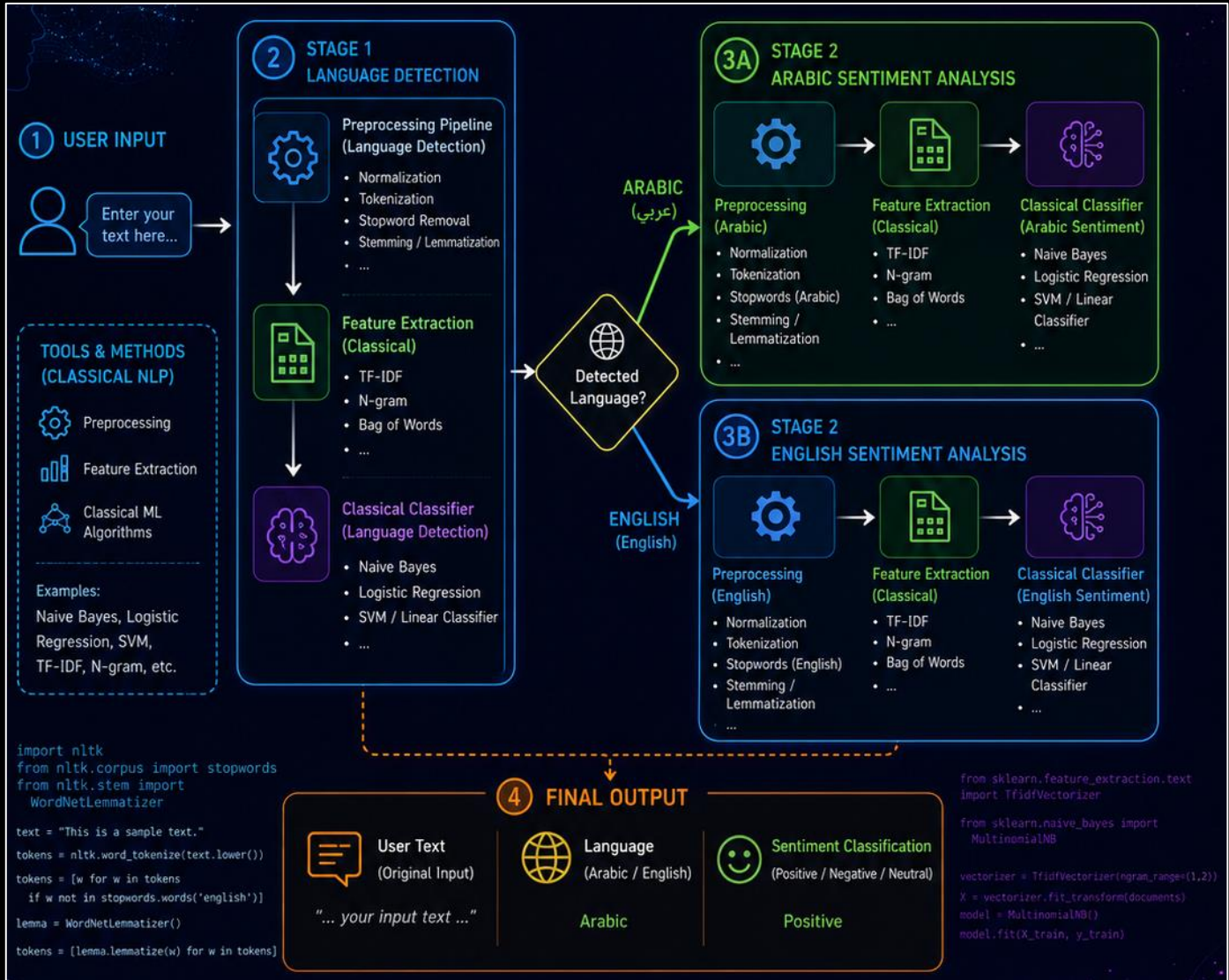
At the end of the complete pipeline, the system must display exactly three main pieces of information:

- **User Text** — The original text entered by the user.
- **Language** — The detected language: Arabic or English.
- **Sentiment Classification** — The sentiment predicted by the corresponding language-specific classifier.

Combined to create a complete multilingual NLP application without relying on modern Transformer or deep-learning solutions.

Project structure

- Preprocessing_pipeline.py
- English_model.py
- Arabic_model.py
- Training_arabic_model.ipynb
- Training_english_model.ipynb
- Training_language_classifier.ipynb
- Arabic_model_weights.pkl
- English_model_weights.pkl
- Language_classifier_weights.pkl





NEUROVA  AI

— AI SOLUTIONS —